

Intern ID : CITS2440

PROJECT TASK - 2

DOCUMENTATION

REPORT

Presented by :

Jahnavi Senagana

4-Bit Ripple Carry Adder

Abstract :

A Ripple Carry Adder is a digital circuit used for binary addition. This project focuses on the design and simulation of a 4-Bit Ripple Carry Adder using Verilog HDL. The circuit performs binary addition of two 4-bit numbers along with carry input and generates sum and carry outputs.

Simulation and waveform analysis were performed using EDA Playground and EPWave.

Introduction :

Binary addition is one of the most important operations in digital electronics and computer architecture. Ripple Carry Adders are widely used arithmetic circuits that perform addition using multiple full adders connected in sequence.

In this project, a 4-Bit Ripple Carry Adder was implemented using Verilog HDL. The carry output from one stage is propagated to the next stage, creating a ripple effect.

Objective :

The objective of this project is to design and simulate a 4-Bit Ripple Carry Adder using Verilog HDL to perform binary addition and verify sum and carry outputs using simulation and waveform analysis.

Tools Used ;

- Verilog HDL
- EDA Playground
- Icarus Verilog Simulator
- EPWave Waveform Viewer

Theory

A Ripple Carry Adder consists of multiple full adders connected in series. Each full adder adds:

- Two input bits
- One carry input

The generated carry output is passed to the next full adder Stage.

Full Adder Equations :

$$\begin{aligned}C_{out} &= \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in} \\&= A B (C_{in} + \overline{C}_{in}) + C_{in} (\overline{A} B + A \overline{B}) \\&= A B + C_{in} (A \oplus B)\end{aligned}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

$$S = A \oplus B \oplus C_{in}$$

Block Diagram

The 4-bit Ripple Carry Adder contains:

- Four Full Adders
- Two 4-bit Inputs
- One Carry Input
- One Carry Output

Each carry propagates sequentially through all stages.

Inputs And Outputs

Inputs :

- A[3:0]
- B[3:0]
- Cin

Outputs :

- Sum[3:0]
- Cout

Verilog Implementation

The project was implemented using:

- Full Adder module
- Ripple Carry Adder module
- Testbench module

The design used assign statements and module instantiation for implementing arithmetic operations.

TestBench

The testbench applies different binary input combinations to verify:

- Correct sum output
- Proper carry propagation
- Functional operation of the adder

Simulation delays were used to observe output transitions.

Simulation

The project was simulated successfully using EDA Playground with Icarus Verilog simulator.

Waveforms were generated using EPWave to verify :

- Input transitions
- Sum outputs
- Carry outputs

Output Waveform Analysis

The waveform output showed:

- Correct binary addition
- Proper carry propagation
- Accurate sum generation

The outputs matched expected binary arithmetic results for all test cases.

TestBench Code, Main Code

The screenshot displays the EDA Playground interface. On the left, there's a sidebar with 'Languages & Libraries' and 'Simulation Options'. The main area is split into two panels: 'testbench.v' and 'design.v'. The 'testbench.v' panel contains a Verilog testbench for a 4-bit Ripple Carry Adder, including input/output declarations, module instantiation, and a final dump command. The 'design.v' panel contains the implementation of a Full Adder module and a 4-bit Ripple Carry Adder module. Below the code panels, a 'Log' tab shows the simulation output, including the command used to run the simulation and the resulting VCD file data.

```
// Testbench for Ripple Carry Adder
module testbench;

// Inputs
reg [3:0] A;
reg [3:0] B;
reg cin;

// Outputs
wire [3:0] SUM;
wire cout;

// Instantiate module
ripple_adder uut(
    .A(A),
    .B(B),
    .cin(cin),
    .SUM(SUM),
    .cout(cout)
);

initial begin
    // Generate waveform
    $dumpfile("dump.vcd");
endendmodule
```

```
// Full Adder Module
module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output carry
);

// Sum operation
assign sum = a ^ b ^ cin;

// Carry operation
assign carry = (a & b) | (b & cin) | (a & cin);

endmodule

// 4-Bit Ripple Carry Adder
module ripple_adder(
    input [3:0] A,
    input [3:0] B,
    input cin,
    output [3:0] SUM,
    output cout
);

full_adder fadd[0];
full_adder fadd[1];
full_adder fadd[2];
full_adder fadd[3];

fadd[0].a = A[0];
fadd[0].b = B[0];
fadd[0].cin = cin;
fadd[0].sum = SUM[0];
fadd[0].carry = fadd[1].cin;

fadd[1].a = A[1];
fadd[1].b = B[1];
fadd[1].cin = fadd[0].carry;
fadd[1].sum = SUM[1];
fadd[1].carry = fadd[2].cin;

fadd[2].a = A[2];
fadd[2].b = B[2];
fadd[2].cin = fadd[1].carry;
fadd[2].sum = SUM[2];
fadd[2].carry = fadd[3].cin;

fadd[3].a = A[3];
fadd[3].b = B[3];
fadd[3].cin = fadd[2].carry;
fadd[3].sum = SUM[3];
fadd[3].carry = cout;

endmodule
```

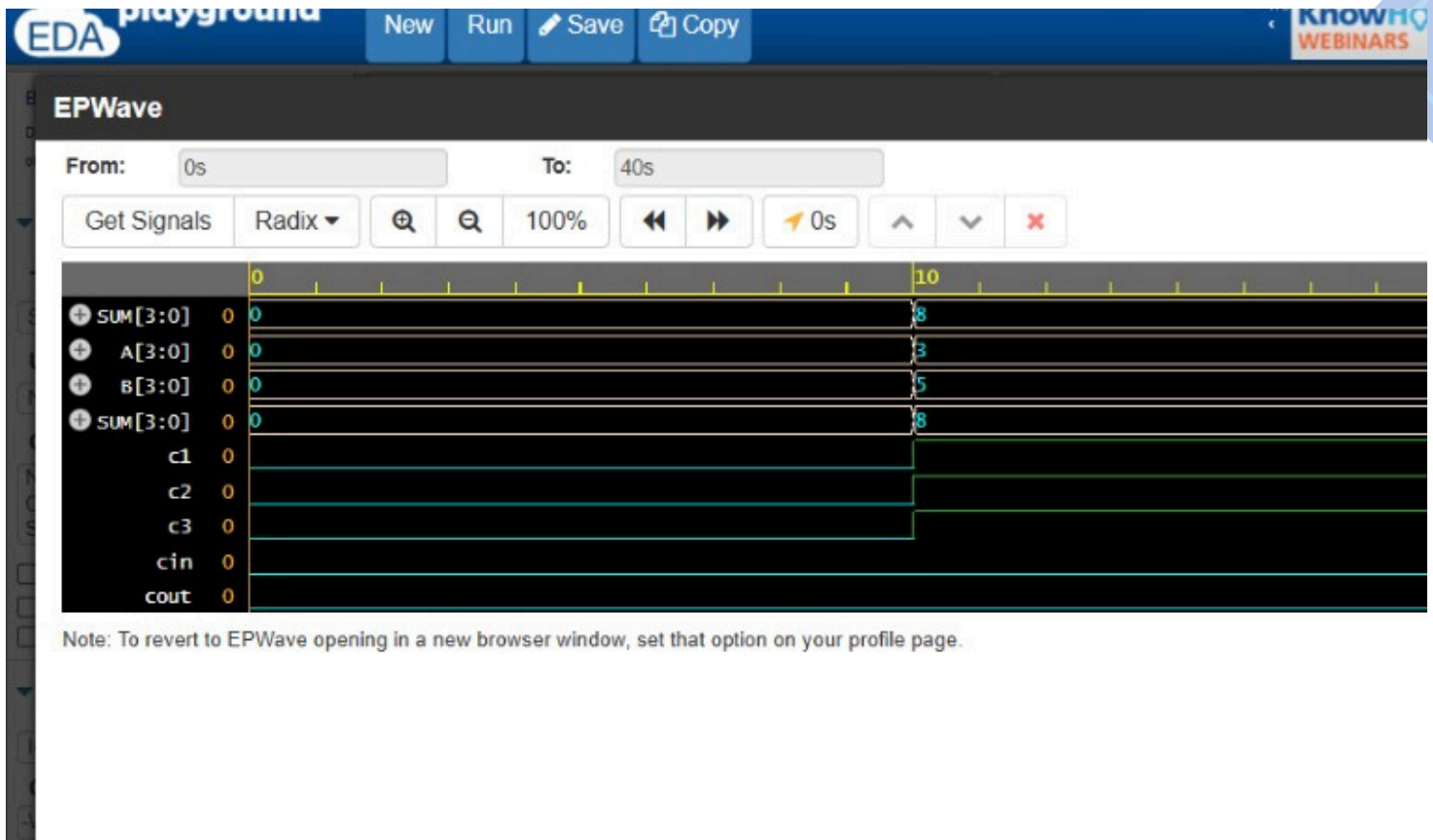
Log

```
[2026-06-04 08:32:04 UTC] iverilog '-wall' design.v testbench.v && unbuffer vvp a.out
VCD info: dumpfile dump.vcd opened for output.
A=0000 B=0000 Cin=0 SUM=0000 Cout=0
A=0011 B=0101 Cin=0 SUM=1000 Cout=0
A=1010 B=0011 Cin=0 SUM=1101 Cout=0
A=1111 B=0001 Cin=0 SUM=0000 Cout=1
testbench.v:62: $finish called at 40 (1s)
Finding VCD file...
./dump.vcd
[2026-06-04 08:32:05 UTC] Opening EPWave...
Done
```

Learning Outcomes

- Understanding binary addition
- Full Adder implementation
- Carry propagation concept
- Verilog HDL programming
- Waveform analysis and simulation

Logic Gates Design-EV Waveforms



References :

- Verilog HDL Documentation
- Digital Electronics Fundamentals
- EDA Playground Simulator Documentation

Conclusion :

The 4-Bit Ripple Carry Adder was successfully designed and simulated using Verilog HDL. The generated outputs matched expected binary addition results, and waveform analysis verified correct carry propagation and arithmetic operation.